LAMP: A Selective-ACK Transport Protocol for Reliable Bulk Data Transmission over LoRa Networks

Alessandro Monticelli

***Abstract*—This paper presents Lightweight Addressed Messaging Protocol (LAMP), a lightweight transport architecture that enables the continuous transfer of large data blocks over point-to-point Long-Range (LoRa®) networks. To overcome the severe Time-on-Air overhead and duty-cycle saturation typical of existing Stop-and-Wait solutions, the proposed architecture segments payloads into up to 255 chunks per transmission window. Each chunk is protected by a 9-byte Logical Header with IPv6-ready node addressing and a 2-byte Cyclic Redundancy Check (CRC)-16 checksum suffix. The protocol provides four distinct operational modes combining connection state (Connectionless / Stateful) and reliability (Best-effort / SACK-ARQ feedback). By requiring receiver feedback only at sequence boundaries rather than per-packet, the Selective Acknowledgment (SACK) mechanism significantly reduces acknowledgment overhead without incurring the constant over-the-air encoding overhead of pure Forward Error Correction (FEC).**

***Index Terms*—Long Range (LoRa), SACK, packet fragmentation, transport protocol, IoT, embedded systems, 3-Way Handshake, point-to-point.**

I. Introduction

LOW-POWER, low-cost, and long-range wireless communication technologies are becoming increasingly important in the context of the Internet of Things (IoT). Applications such as environmental monitoring, precision agriculture, and autonomous Unmanned Aerial Vehicles (UAVs) require reliable data transmission over extended distances under strict power and hardware constraints [1].

Several communication standards, including ZigBee, Sigfox, and Long Range (LoRa) have been adopted to address these needs, and LoRa, in particular, has emerged as the leading choice for low-power and long-range deployments thanks to its robust modulation technique and open network architecture [2]. However, conventional solutions based on LoRa (particularly those following the LoRaWAN® specifications) are optimized for small, non-frequent payloads [3].

This limitation poses severe restrictions on data-intensive IoT applications which demand the continuous transfer of large data blocks [4], [5], such as Firmware Update Over The Air (FUOTA), Simultaneous Localization and Mapping (SLAM) map transfers in autonomous robotics, or bulk sensor log collection. Standard Long Range Wide Area Network (LoRaWAN) links require substantial transmission overhead, restrictive duty-cycle limitations [6], and lack optimized mechanisms for handling large, multi-packet payloads in point-to-point configurations.

This paper presents the design and implementation of LAMP, a lightweight transport protocol for point-to-point and local multi-node LoRa networks. LAMP supports payloads up to 62.2 KB on 255-byte-First-In, First-Out (FIFO) transceivers (such as the Semtech® SX1262), operating independently of higher-layer middleware such as LoRaWAN.

The main contributions of this work are:

- **Bitmap-Based SACK Scheme**: Unlike existing Low-Power Wide-Area Network (LPWAN) fragmentation solutions that rely on Stop-and-Wait (S&W) Automatic Repeat Request (ARQ), which requires one acknowledgment per packet and incurs severe latency under duty-cycle constraints, the proposed protocol uses a compact binary bitmap carried in a single feedback frame. The receiver reports the delivery status of up to 255 chunks per round-trip, reducing retransmission overhead and over-the-air footprint.
- **Lightweight Packet Segmentation and Abstraction**: The protocol introduces a 9-byte logical header (with 8-bit IPv6-ready node addressing) and a custom CRC integrity check on partial payloads, enabling out-of-order chunk reassembly on resource-constrained microcontrollers.
- **Modular C++17 Implementation and Test-Driven Development (TDD) Validation**: The core state machine is completely decoupled from the physical radio driver through a Hardware Abstraction Layer (HAL). This design enables both native host-based unit testing (using the Unity framework) and physical deployment on Semtech SX126x transceivers.

The remainder of this paper is organized as follows. Section II surveys related work and defines the design space. Section III details the system architecture, packet structure, and formal protocol algorithms. Section IV outlines preliminary prototyping findings. Section V presents the hardware-level C++17 implementation. Section VI evaluates performance through extensive ns-3 simulations and planned field validation. Section VII concludes the paper and presents the architectural roadmap.

II. State of the Art

Reliable packet fragmentation and reassembly over LPWANs has been the subject of active research, driven by the growing demand for data-intensive IoT applications.

A prominent industrial standard is Static Context Header Compression (SCHC) [7], which introduces a framework for

fragmenting IPv6 packets over resource-constrained networks. To guarantee reliability, SCHC ACK frames carry a bitmap representing the status of individual tiles within a window (ACK-on-Error). However, SCHC is designed as a generic adaptation layer for IP-based architectures, requiring pre-shared context rules and gateway-centric networks. Similarly, the LoRa Alliance standardized the Fragmented Data Block Transport specification to support FUOTA using FEC codes to reconstruct blocks without feedback. However, FEC-based recovery introduces a constant over-the-air encoding overhead and high computational requirements for low-cost embedded systems.

For custom peer-to-peer LoRa networks, early efforts focused on overcoming the severe airtime latencies and capacity constraints of traditional Stop-and-Wait (S&W) ARQ schemes. Empirical link evaluations by Cattani et al. [8] and network scalability analyses by Bor et al. [9] demonstrated that high packet-loss rates and strict Time-on-Air limits make per-packet Stop-and-Wait acknowledgments unviable for bulk data transfers [4]. Chen et al. [10] proposed the Multi-Packet LoRa (MPLR) protocol, introducing the concept of replacing Stop-and-Wait ARQ with a windowed batch transmission scheme acknowledged by a bit-vector acknowledgment packet (`BVACK`). Subsequent specialized studies explored multi-hop relay networks for visual IoT coverage [11], comprehensive mesh survey frameworks [12], and Time Division Multiplexing (TDM) scheduled channels [13]. As a pure transport-layer protocol, LAMP is topology-agnostic and can operate transparently on top of multi-hop mesh routing protocols to extend reliable bulk delivery across multi-hop topologies.

Building on the bit-vector SACK concept, LAMP extends and generalizes this approach with several key protocol-level optimizations:

- **Header and Protocol Overhead Optimization**: MPLR relies on a 16-byte header per fragment (carrying 4-byte Destination EUI, 4-byte Source EUI, Service Number, Sequence Number, Flags, Payload Size, Batch Size, and Checksum). In contrast, LAMP utilizes a 9-byte Logical Header (1-byte `srcAddr`, 1-byte `dstAddr`, 2-byte `messageId`, 1-byte `totalChunks`, 1-byte `chunkIndex`, 1-byte `payloadSize`, 1-byte `flags`, and 1-byte `protocolVersion`). Including the 2-byte CRC-16 checksum suffix, LAMP requires a total over-the-air overhead of 11 bytes per frame compared to MPLR's 16 bytes, yielding a 5-byte reduction in total over-the-air overhead per fragment.
- **Target Scope**: MPLR was engineered specifically as an application-level image transmission protocol for agricultural uplink nodes to a central gateway. LAMP is designed as a domain-agnostic, general-purpose transport layer capable of segmenting any arbitrary data payload up to 62.2 KB on standard LoRa transceivers (ranging from high-frequency sensor telemetry arrays and robotic SLAM keyframes to firmware binaries and multiagent state matrices).
- **Flexible Operational Modes**: MPLR operates under a single stateful batch delivery model. In contrast, LAMP provides four operational modes spanning *Unreliable Connectionless* (best-effort datagrams), *Unreliable Connected* (session-tracked streaming), *Reliable Connectionless* (stateless SACK-ARQ), and *Reliable Connected* (stateful negotiated streams).
- **Dynamic Parameter Negotiation**: MPLR utilizes a fixed batch size and requires a 4-way handshake tear-down (`FIN`/`ACK`). LAMP implements a lightweight 3-Way Handshake (3WHS) embedded with dynamic parameter negotiation (`SynMetadata` and `SynAckMetadata` negotiating chunk size, sliding window, and inactivity timeout) and replaces explicit tear-down overhead with automatic idle session pruning.

To clearly position the proposed protocol within the current technological landscape and define the precise design space in which LAMP operates, Table I provides an architectural comparison with other prominent fragmentation and transport solutions over LoRa, including Stop-and-Wait ARQ, SCHC [7], LoRaWAN Fragmented Data Block Transport, and MPLR [10].

Rather than claiming overall superiority over existing industrial standards, LAMP is designed to occupy a specific, previously unaddressed architectural niche: providing a *lightweight, autonomous, and highly configurable transport layer for raw LoRa links*. Each protocol in Table I represents a distinct set of engineering trade-offs tailored to specific deployment constraints:

- **SCHC [7]**: Represents the de facto standard for IP-connected star networks, achieving header compression down to 1–2 bytes for standard IPv6/UDP traffic. However, it relies on gateway-centric infrastructure and pre-shared static context rules, making it less suitable for standalone, ad-hoc, raw-LoRa peer-to-peer or multi-node links without gateway context managers.
- **LoRaWAN Fragmented Transport**: Optimized for unidirectional downlink firmware updates (FUOTA) to thousands of nodes simultaneously using Forward Error Correction (FEC) without requiring a feedback channel. However, pure FEC introduces constant over-the-air encoding overhead regardless of channel quality and imposes significant computational and memory decoding overhead on low-cost microcontrollers.
- **MPLR [10]**: Engineered specifically for high-volume agricultural image uplink transfers (e.g., 9 KB to multi-megabyte images) using 40-packet batching with bit-vector acknowledgments over a 16-bit sequence space (up to 65,536 distinct message IDs). However, it utilizes a 16-byte fixed header (8-byte EUI addressing), operates under a single stateful batch mode, and requires explicit 4-way handshake teardown (`FIN`/`ACK`).
- **Stop-and-Wait ARQ**: Offers minimal header size (2–4 bytes) and trivial stateless implementation for isolated single-packet telemetry. However, it incurs prohibitive airtime latencies and severe ETSI duty-cycle saturation when applied to multi-fragment payloads due to per-packet ACK waiting delays.
- **LAMP (Proposed Protocol)**: Fills the design space of standalone raw-LoRa links by providing four con-

figurable operational modes (Reliable/Unreliable × Connected/Connectionless), 8-bit local unicast/broadcast addressing, dynamic 3WHS buffer negotiation, and transport-layer purity (decoupled from opt-in HAL channel access helpers like CAD/CSMA-CA). *Protocol Limitations & Trade-offs*: To optimize header efficiency, LAMP allocates 1 byte for fragment indexing (`totalChunks` and `chunkIndex`), bounding a single SACK bitmap session to 255 chunks (62.2 KB total payload size, requiring sequential sessions for larger files). Furthermore, carrying 8-bit node addresses and sequence metadata requires a 9-byte Logical Header (+2-byte CRC-16 suffix, totaling 11 bytes overhead per frame), and session tracking requires microcontroller RAM allocation for SACK bitmaps (dynamically sized up to 32 bytes for 255 chunks, $\lceil \text{totalChunks}/8 \rceil$) and millisecond timer primitives, unlike stateless Stop-and-Wait ARQ.

III. Proposed System Architecture

LAMP is a transport-layer protocol running directly over the LoRa physical layer. It segments arbitrarily large payloads (within the described constraints) at the transmitter and reconstructs them at the receiver while minimizing Logical Header and Over-the-Air (OTA) overhead.

A. Protocol Specification and Design Iterations

We describe the design of LAMP by contrasting its initial stateless baseline (LAMPv1) with the current addressed architecture (LAMPv2).

1) Stateless Header Specification: The initial protocol version, LAMPv1, was designed for maximum header compression during periodic sensor telemetry transfers over point-to-point links. It utilized a compact 7-byte Logical Header containing only sequence tracking and payload bounds (`messageId` [2 B], `totalChunks` [1 B], `chunkIndex` [1 B], `payloadSize` [1 B], `flags` [1 B], and `protocolVersion` [1 B]).

While LAMPv1 achieved high spectral efficiency for isolated datagram bursts, deployment on embedded hardware revealed three key architectural limitations:

- *Speculative Buffer Allocation*: Receivers allocated reassembly memory speculatively upon receiving the first fragment of an unannounced stream. When receiving large payloads (e.g., image tiles or firmware blocks), memory-constrained microcontrollers risked heap exhaustion or buffer overflows.
- *Session Interference*: Lacking explicit source and destination identifiers, multiple unaddressed transmitters sharing the same RF channel risked interleaving fragments with identical sequence numbers, leading to corrupted reassembly buffers.
- *Static Transport Limits*: Fixed chunk sizes and static timeout thresholds prevented dynamic adaptation to link quality degradation or variable receiver memory limits.

2) Addressed Logical Header Specification: To resolve the limitations of LAMPv1, LAMPv2 introduces an addressed, stateful transport architecture. LAMPv2 expands the Logical Header to 9 bytes by embedding explicit 8-bit Source (`srcAddr`) and Destination (`dstAddr`) Node Identifiers, while incorporating an integrated 3WHS mechanism.

Although adding 2 bytes to the header increases over-the-air frame length by less than 0.8% relative to the maximum 255-byte frame (9-byte header + 244-byte payload + 2-byte CRC-16), this trade-off provides three main architectural benefits:

1) *Buffer Memory Safety*: The 3WHS exchange allows nodes to negotiate reassembly buffer limits (`SynMetadata`) prior to payload transmission, preventing allocation failures on memory-constrained receivers.
2) *Channel Session Isolation*: Explicit node addressing prevents session crosstalk and frame interleaving in multi-node wireless topologies.
3) *IPv6 Integration Readiness*: The 8-bit Node ID introduces a lightweight local addressing namespace that can be mapped directly to an IPv6 link-local prefix at an edge gateway, enabling transparent IP-layer integration without transmitting heavy 128-bit IPv6 headers over raw LoRa radio links.

B. Operational Modes and Sequence Management

To accommodate diverse IoT workload requirements, LAMP defines four distinct operational modes across two primary dimensions: *Reliability* (Unreliable Best-Effort vs. SACK-ARQ) and *Connection State* (Connectionless Handshake-Free vs. Connection-Oriented Stateful).

1) Implicit Frame Boundary Derivation: To maintain a compact 9-byte header while supporting robust out-of-order packet reassembly, LAMP avoids dedicated boundary control bits. Instead, Start-of-Message (SOM) and End-of-Message (EOM) boundaries are implicitly derived at the receiver directly from the 0-based fragment index (`chunkIndex`) and the total fragment count (`totalChunks`):

$$\text{SOM} \iff \texttt{chunkIndex} == 0 \tag{1}$$

$$\text{EOM} \iff \texttt{chunkIndex} == \texttt{totalChunks} - 1 \tag{2}$$

Explicitly carrying `totalChunks` in every fragment header allows the receiver to determine the exact message bounds and pre-allocate the required reassembly buffer and SACK bitmap upon receiving *any* fragment, even if fragments arrive out of order or if the initial chunk is lost.

2) Node Addressing Scheme: To support multi-node communication without incurring the prohibitive airtime overhead of full 128-bit IPv6 headers on LoRa PHY payloads, LAMP introduces a compact 8-bit Node Identifier space:

- **`0x00` (`ADDRESS_UNASSIGNED`)**: Reserved for legacy or unaddressed P2P datagrams (promiscuous mode).
- **`0x01`–`0xFE`**: 254 unique individual node addresses (Unicast).
- **`0xFF` (`ADDRESS_BROADCAST`)**: Broadcast address processed by all active receiving nodes on the RF channel.

TABLE I
ARCHITECTURAL COMPARISON AND TRADE-OFF MATRIX OF LORA TRANSPORT SOLUTIONS

Architectural Dimension	**Stop-and-Wait ARQ**	**SCHC [7]**	**LoRaWAN Fragmented**	**MPLR [10]**	**Proposed (LAMP)**
Stack Dependency	Raw LoRa PHY	Full LoRaWAN / IP Stack	Full LoRaWAN Stack	Raw LoRa PHY	Raw LoRa PHY (Standalone)
ACK Mechanism	Per-Packet ACK	ACK-on-Error Bitmap	None (FEC Code)	Bit-Vector SACK	Bit-Vector SACK
Header Overhead	2–4 Bytes	1–2 Bytes (Compressed)	2 Bytes	16 Bytes	9 Bytes (11B total w/ CRC)
Max Frame Payload	240+ Bytes	Variable	Variable	239 Bytes	244 Bytes (255B PHY limit)
Max Session Payload	Unbounded (Sequential)	Variable (IPv6 Packet)	Unlimited (Block Session)	Multi-MB Image (65k Packets)	62.2 KB (255 Chunks / Session)
Operational Modes	1 (Reliable S&W)	2 (ACK / No-ACK)	1 (Unreliable FEC)	1 (Reliable Batch)	4 Configurable Modes
Node Addressing	None / Application	IPv6 128-bit (Compressed)	DevAddr (32-bit)	EUI (64-bit)	8-bit Unicast + Broadcast
Multi-Node Suitability	Single P2P Link	Star (Gateway-Centric)	Star Broadcast (Downlink)	P2P Uplink (Node–GW)	P2P & Multi-Node Ready
Parameter Negotiation	None (Static)	Static (Pre-shared Rules)	Static / Out-of-Band	Static Batch / 4WHS	Dynamic 3WHS Metadata
Key Strengths	Minimal header size, trivial implementation	Extreme header compression for IP traffic	No feedback channel required (pure FEC)	High-volume image uplink batching	4 flexible modes, dynamic 3WHS buffer safety, raw LoRa autonomy
Key Limitations & Trade-offs	Prohibitive airtime latency, duty-cycle saturation	Requires static gateway rules & IP core infrastructure	High FEC compute cost, constant airtime overhead	16B header overhead, single mode, 4WHS teardown	9B header (11B w/ CRC), 62.2 KB session limit, RAM/timer state

TABLE II
LAMP OPERATIONAL MODES ARCHITECTURE

Operational Mode	**Reliability & Connection State**	**Primary Target Use Case**
Mode 1: Best-Effort Datagram	Unreliable, Connectionless (Stateless)	High-frequency telemetry streams, periodic sensor data bursts.
Mode 2: Best-Effort Stream	Unreliable, Connection-Oriented (Stateful)	Session-tracked real-time sensor streams without retransmission delays.
Mode 3: SACK-Reliable Datagram	Reliable (SACK), Connectionless (Handshake-Free)	Isolated high-priority sensor bursts requiring delivery confirmation without prior session setup.
Mode 4: SACK-Reliable Stream	Reliable (SACK), Connection-Oriented (3WHS)	Critical large payload transfers (FUOTA binaries, camera images, SLAM maps).

This 8-bit Node ID provides a lightweight local namespace: at an edge gateway or application-layer bridge, a local 8-bit Node ID (e.g., `0x0A`) can be mapped to an IPv6 link-local subnet suffix (e.g., `fe80::0a`), enabling transparent IP-layer integration without transmitting heavy IPv6 headers over the constrained radio link.

The 8-bit address space supports up to 254 unicast nodes per RF cluster, which is sufficient for the dense but bounded deployments targeted by this work (e.g., agricultural sensor arrays, UAV swarm sub-clusters). For applications requiring broader network reach, Section VII foresees integration of full IPv6 routing, where the 8-bit node identifier serves as a local cluster suffix rather than a global unique identifier. Note that the 254 addressable nodes operate independently of the receiver's concurrency limit (`MAX_CONCURRENT_SESSIONS` $= 10$), which governs the maximum number of simultaneously active reassembly sessions.

The exact byte alignment and offsets of the 9-byte header packet structure are presented in Table III and Figure 1.

TABLE III
LAMP OTA PACKET LAYOUT AND BYTE OFFSETS

Offset	**Field Name**	**Type**	**Description**
0	`srcAddr`	uint8_t	Source Node ID (`0x01`–`0xFE`)
1	`dstAddr`	uint8_t	Destination Node ID (`0x01`–`0xFE`, `0xFF`)
2–3	`messageId`	uint16_t	Message Sequence ID
4	`totalChunks`	uint8_t	Total Count ($1 \leq N \leq 255$; $N = 0$ invalid)
5	`chunkIndex`	uint8_t	Index ($0 \leq i < N$)
6	`payloadSize`	uint8_t	Chunk Payload ($1 \leq L \leq 244$ B on SX1262)
7	`flags`	uint8_t	Bitfield (`ACK_REQ`, `CONN_REQ`)
8	`protocolVersion`	uint8_t	Protocol Version (`0x02`)
9–L+8	`payload[]`	uint8_t[]	Application Data (L bytes)
L+9–L+10	`crc`	uint16_t	CRC-16 Checksum

Byte 0	Byte 1	Byte 2 – 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8
srcAddr	dstAddr	messageId	totalChunks	chunkIndex	payloadSize	flags	protocolVersion

Fig. 1. Byte layout of the compact 9-byte Logical Header incorporating Node Addressing.

The fields in the Logical Header are defined as follows:

- **srcAddr (1 byte)**: Source node identifier (0x01–0xFE, or 0x00 for unassigned).
- **dstAddr (1 byte)**: Destination node identifier (0x01–0xFE, or 0xFF for broadcast).
- **messageId (2 bytes)**: Unique message sequence identifier. All fragments composing the same segmented message share this ID.
- **totalChunks (1 byte)**: The total count of chunks into which the original large payload is divided ($1 \leq$ totalChunks ≤ 255).
- **chunkIndex (1 byte)**: The 0-based index of this specific fragment ($0 \leq$ chunkIndex $<$ totalChunks).
- **payloadSize (1 byte)**: Payload data byte count. Logically, the 1-byte field supports up to 255 bytes per chunk, enabling a theoretical protocol capacity of $255 \times 255 = 65,025$ bytes (≈ 65.0 KB) per session. On standard transceivers with a 255-byte hardware FIFO limit (e.g., SX1262), subtracting the 9-byte header and 2-byte CRC-16 yields $N = 244$ bytes per chunk, resulting in an empirical capacity of $255 \times 244 = 62,220$ bytes (≈ 62.2 KB).
- **flags (1 byte)**: A control bitfield used to manage transmission and connection states:
  - FLAG_ACK_REQ (0x04): Selective Acknowledgment requested from receiver.
  - FLAG_ACK (0x08): SACK feedback packet containing binary bitmap of received chunks.
  - FLAG_CONN_REQ (0x10): Connection Request control frame.
  - FLAG_CONN_ACK (0x20): Connection Acknowledged control frame.
  - FLAG_CONN_NACK (0x40): Connection Refused control frame.
- **protocolVersion (1 byte)**: Identifies protocol version (currently 2) to ensure backward compatibility.

After the payload, a 16-bit CRC-16 (Modbus CRC-16, polynomial 0xA001, initialized to 0xFFFF) is appended. The checksum is computed over the concatenation of the 9-byte Logical Header and exactly payloadSize application-layer bytes; any trailing buffer bytes beyond payloadSize are excluded, so that partial final chunks are validated identically to full-size chunks and inter-implementation interoperability is preserved.

Table IV summarises all defined flag combinations and the behaviour of the packet parser upon receipt of an undefined value.

C. Selective Acknowledgment and Retransmission

In reliable mode, the protocol implements a SACK mechanism [14] to recover lost fragments with minimal additional airtime.

TABLE IV
LAMP FLAG FIELD: VALID COMBINATIONS AND PARSER BEHAVIOUR

Value	Semantic	Notes
0x00	Data chunk	Unreliable/reliable chunk, no ACK
0x04	Final chunk (ACK_REQ)	Receiver replies with SACK
0x08	SACK feedback	Payload = received-chunk bitmap
0x10	SYN	3WHS Step 1 (SynMetadata)
0x30	SYN-ACK	3WHS Step 2 (SynAckMetadata)
0x20	ACK	3WHS Step 3, no payload
0x40	CONN-NACK	Payload = rejection reason code
other	*Undefined*	Silently discarded

1) Retransmission Protocol Interaction: When a transmission is executed in reliable mode:

1) The transmitter segments the payload, serializes the chunks with srcAddr and dstAddr, and streams them sequentially. The final chunk index ($Index =$ totalChunks $- 1$) is marked with the FLAG_ACK_REQ flag.
2) Upon receipt of the chunk requesting acknowledgment, the receiver evaluates its reassembly session. It identifies which fragments are still missing by checking its internal bitmask of received chunks.
3) The receiver responds by transmitting a SACK packet (FLAG_ACK) addressed back to the transmitter's srcAddr. The SACK's payload contains a binary bitmap where each bit represents the receipt status of a chunk (1 for received, 0 for missing).
4) The transmitter parses the received SACK bitmap. If some chunks are missing, it selectively retransmits only those lost chunks. The last retransmitted chunk has the FLAG_ACK_REQ flag set again to query the receiver.
5) This selective retry loop continues until the full message is reconstructed at the receiver, or the transmitter reaches the maximum retry count.

2) Half-Duplex Hardware Switching Guard: Because LoRa transceivers are half-duplex, a hardware transition period is required for the transmitter to switch from transmit (TX) mode to receive (RX) mode after sending a packet. While physical SX1262 transceiver state transitions take $< 100\,\mu$s, the $\delta = 25$ ms guard delay accounts for physical transceiver switching alongside software-level Real-Time Operating System (RTOS) task scheduling jitter, Serial Peripheral Interface (SPI) bus transaction queuing, and microcontroller wake-up latencies on embedded targets.

It is important to emphasize that all protocol timing parameters, guard delays (e.g., SACK_PREAMBLE_GUARD_DELAY_MS), and retry limits are fully decoupled into a centralized configuration namespace (LoRaMultiPacketConfig) and can be customized by integrators for any MCU/radio target. The default parameters presented in this study were specifically tuned around the Heltec Wi-Fi LoRa 32 V3 (ESP32-S3) hardware platform.

3) Timeout and Fallback Recovery: If the transmitter does not receive a SACK packet within the dynamic timeout window, it assumes the query or response was lost and retransmits the last `FLAG_ACK_REQ`-marked chunk. The transmitter permits up to $R_{\max} = 5$ consecutive timeout events before aborting the session.

The timeout is computed adaptively as:

$$T_{\text{timeout}} = \max\Big(T_{\min},\ \alpha \cdot \widehat{T}_{\text{oA}} + \delta\Big) \tag{3}$$

where $T_{\min} = 1000$ ms is the minimum guard, $\alpha = 1.5$ is a safety factor, $\widehat{T}_{\text{oA}}$ is the Time-on-Air for the current packet length queried from the radio driver [15], and $\delta = 25$ ms is the half-duplex switching and scheduling guard delay. Table V reports representative timeout values across spreading factors.

4) Analytical Burst Reliability Bounding: Under a frame loss probability p, the probability that a specific chunk or SACK query fails after $R_{\max}$ retransmission retries is $p^{R_{\max}+1}$. Consequently, for a data payload segmented into N chunks, the overall burst delivery success probability P_{success} and overall burst failure probability P_{fail} under SACK-ARQ are analytically bounded by:

$$P_{\text{success}} = \left(1 - p^{R_{\max}+1}\right)^N, \qquad P_{\text{fail}} = 1 - P_{\text{success}} \tag{4}$$

For a lossy wireless channel with a 20% frame error rate ($p = 0.20$) and the default retry limit $R_{\max} = 5$ (configured via `LoRaMultiPacketConfig::MAX_RETRIES`), the individual chunk failure probability drops to $p^6 = 0.000064$, yielding an overall burst delivery success probability $P_{\text{success}} = (1 - 0.000064)^{20} \approx 99.87\%$ for a standard 20-chunk payload transfer.

TABLE V
ADAPTIVE SACK TIMEOUT VALUES (BW 500 KHZ, CR 4/5, 255-BYTE MAX FRAME)

SF	$\widehat{T}_{\text{oA}}$ (ms)	$\alpha \cdot \widehat{T}_{\text{oA}} + \delta$ (ms)	T_{timeout} (ms)
7	99	174	1000
8	174	286	1000
9	313	494	1000
10	564	871	1000
11	1025	1563	1563
12	1886	2854	2854

D. Memory Bounding and RAM Footprint Analysis

To guarantee memory safety on constrained microcontrollers, the dynamic memory allocated for an active reassembly session at the receiver is strictly bounded by:

$$M_{\text{session}} = S_{\text{struct}} + N_{\text{chunks}} \cdot S_{\text{chunk}} + \left\lceil \frac{N_{\text{chunks}}}{8} \right\rceil \tag{5}$$

where $S_{\text{struct}} \approx 48$ bytes represents the session tracking structure metadata, $S_{\text{chunk}} \leq 244$ bytes is the maximum payload chunk size, and the final term represents the binary SACK bitmask.

For a representative 20-chunk payload (4.88 KB data size), a complete reassembly session requires only $\approx$ 4.93 KB of RAM. Furthermore, the protocol bounds the maximum number of concurrent active sessions (`MAX_CONCURRENT_SESSIONS` = 10), placing an absolute upper bound of $\approx$ 49.3 KB on total protocol RAM consumption, making it well-suited for low-cost IoT microcontrollers with limited SRAM capacities.

E. Formal Protocol Algorithms

To ensure complete specification clarity and system-level reproducibility across different hardware platforms, Algorithm 1 and Algorithm 2 formally detail the transmitter-side segmentation pipeline and the receiver-side out-of-order reassembly engine, respectively.

F. Session Control and Handshake Protocol

To support stateful stream negotiation and protect resource-constrained receivers from buffer overflows, LAMPv2 introduces a 3WHS mechanism.

1) Three-Way Handshake Procedure: As illustrated in Figure 3(a), the connection lifecycle follows a three-step exchange:

1) **SYN**: The initiator sends a `SYN` control frame (`FLAG_CONN_REQ`) addressed to the target node's `dstAddr`, containing a 4-byte `SynMetadata` payload (`requestedPayloadSize`, `windowSize`, `timeoutMs`).
2) **SYN-ACK**: Upon receiving the `SYN`, the peer evaluates available memory and protocol version. If accepted, it responds with a `SYN-ACK` control frame (`FLAG_CONN_REQ | FLAG_CONN_ACK`) addressed back to the initiator. The receiver clamps `acceptedPayloadSize` to its maximum buffer capability.
3) **ACK**: The initiator parses the `SYN-ACK`, adopts the negotiated payload size, and transmits a final `ACK` frame (`FLAG_CONN_ACK`). Both nodes transition their internal state machine to `ESTABLISHED`.

G. Concurrency and Fault Handling

To ensure protocol stability in multi-node and lossy environments, LAMP incorporates the following fault-handling mechanisms:

- *Simultaneous Connection Resolution*: If two nodes transmit `SYN` frames to each other simultaneously, a deterministic tie-breaking rule resolves the conflict: the node with the lower numerical `NodeAddress` yields and transitions back to `CLOSED`, while the node with the higher `NodeAddress` proceeds with session setup.
- *Receiver Busy Rejection*: If a receiver receives a `SYN` frame while its active reassembly session is already locked by another node, it responds with a `CONN_NACK` frame (`FLAG_CONN_NACK`) specifying a `BUSY` status code, causing the requester to back off.
- *Source Address Filtering*: Initiators in `waitForSynAck` and `waitForSack` validate incoming control frames against the expected `srcAddr`, ignoring packets from unrelated third-party nodes.

Algorithm 1 LAMP Payload Transmission and SACK-ARQ (Transmitter)

Require: Payload P (size L), $dstAddr$, Max Chunk Size S, Mode M
Ensure: Delivery Status (`OK` or `FAIL`)
$N \leftarrow \lceil L/S \rceil$
if $N > 255$ **then**
  return `ERR_PAYLOAD_TOO_LARGE`
end if
$msgId \leftarrow$ GetNextMsgId()
for $i \leftarrow 0$ **to** $N-1$ **do**
  $cLen \leftarrow (i == N-1)?(L - i \cdot S) : S$
  $flags \leftarrow (M \in \{\text{M3}, \text{M4}\}$ **and** $i == N-1)?$`FLAG_ACK_REQ` : `0x00`
  $hdr \leftarrow$ BuildHdr$(src, dst, msgId, N, i, cLen, flags)$
  $crc \leftarrow$ CalcCRC16$(hdr, P[i \cdot S : i \cdot S + cLen])$
  $F_i \leftarrow$ SerializeFrame$(hdr, P[i \cdot S : i \cdot S + cLen], crc)$
  Transmit(F_i)
end for
if $M \in \{\text{Mode } 1, \text{Mode } 2\}$ **then**
  return `OK` ▷ Best-effort datagram/stream
end if
$retries \leftarrow 0$
while $retries < R_{\max}$ **do**
  $sack \leftarrow$ WaitForSack(T_{timeout})
  if $sack ==$ TIMEOUT **then**
    Transmit(F_{N-1}); $retries \leftarrow retries + 1$
  else if IsAllAcked$(sack.bitmap)$ **then**
    return `OK` ▷ Complete payload delivery confirmed
  else
    for $idx \in$ MissingChunks$(sack.bitmap)$ **do**
      Transmit(F_{idx})
    end for
  end if
end while
return `ERR_MAX_RETRIES_EXCEEDED`

Algorithm 2 Out-of-Order Reassembly and SACK Generation (Receiver)

Require: Received Frame F
Ensure: Reassembled Payload P or Drop Status
$(hdr, payload, crc) \leftarrow$ ParseFrame(F)
if ValidateCRC$(hdr, payload, crc) ==$ FAIL **then**
  Drop(F); **return**
end if
if $hdr.dstAddr \neq myAddr$ **and** $hdr.dstAddr \neq$ `0xFF` **then**
  Drop(F); **return**
end if
if $hdr.totalChunks == 0$ **or** $hdr.chunkIndex \geq hdr.totalChunks$ **then**
  Drop(F); **return**
end if
$session \leftarrow$ FindSession$(hdr.srcAddr, hdr.msgId)$
if $session ==$ NULL **then**
  if AvailMem() $< hdr.totalChunks \cdot hdr.payloadSize$ **then** ▷ Safety check
    SendNack(hdr.srcAddr, `BUSY`); **return**
  end if
  $session \leftarrow$ AllocBuf$(hdr.totalChunks, hdr.payloadSize)$
end if
$idx \leftarrow hdr.chunkIndex$
if IsBitSet$(session.bitmap, idx)$ **then**
  Drop(F) ▷ Duplicate chunk, ignore write
else
  WriteChunk$(session.buffer, idx, payload)$
  SetBit$(session.bitmap, idx)$
  $session.count \leftarrow session.count + 1$
end if
if $hdr.flags$ & `FLAG_ACK_REQ` **then**
  SendSack$(hdr.srcAddr, hdr.msgId, session.bitmap)$
end if
if $session.count == hdr.totalChunks$ **then**
  $P \leftarrow$ ExtractPayload$(session.buffer)$
  DeliverToApp(P); FreeSession$(session)$
end if

- *Incomplete Handshake Timeout*: If a `SYN-ACK` or `ACK` of a 3WHS is lost in transit, the receiver in `SYN_RCVD` or initiator in `SYN_SENT` automatically aborts and resets to `CLOSED` after an inactivity timeout (`timeoutMs`).

IV. Preliminary Proof of Concept

An initial prototype using commercial Ebyte E220-series modules validated the core segmentation concept. Field trials confirmed feasibility but exposed three limitations: (1) the modules exposed a proprietary network layer that prevented direct control of physical-layer parameters; (2) a large, unoptimized application header inflated per-chunk airtime; and (3) no acknowledgment or retransmission mechanism was available.

These findings motivated the redesign on Semtech SX126x-class transceivers via the RadioLib driver, validated on Heltec Wi-Fi LoRa 32 V3 boards. The revised implementation described in the following sections, is fully hardware-agnostic through the HAL interface and incorporates the 9-byte Logical Header, CRC-16 integrity checks, and the SACK-based retransmission mechanism.

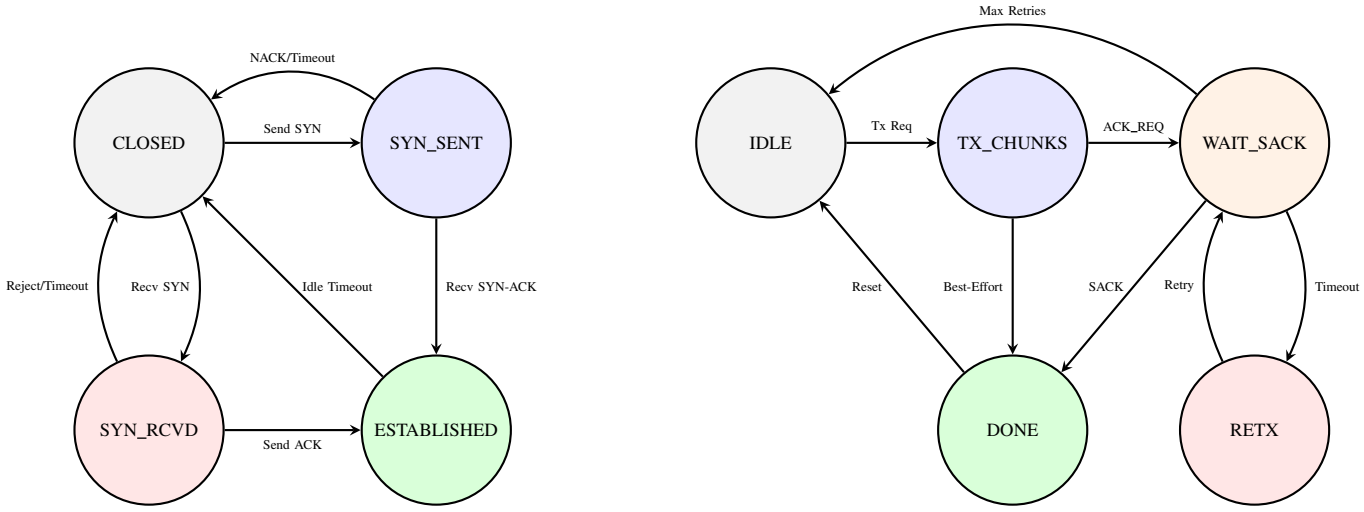
V. Hardware Level Implementation

To validate the proposed protocol, a complete implementation of the LAMP library was developed in C++17. The design is modular, separating the transport state machine from physical hardware dependencies and parameter policies.

A. Centralized Configuration and Parameter Decoupling

To eliminate hardcoded constants and enable deterministic tuning across heterogeneous embedded targets, all protocol operational parameters are centralized within a dedicated compile-time configuration namespace (`LoRaMultiPacketConfig`). This design encapsulates three primary domains:

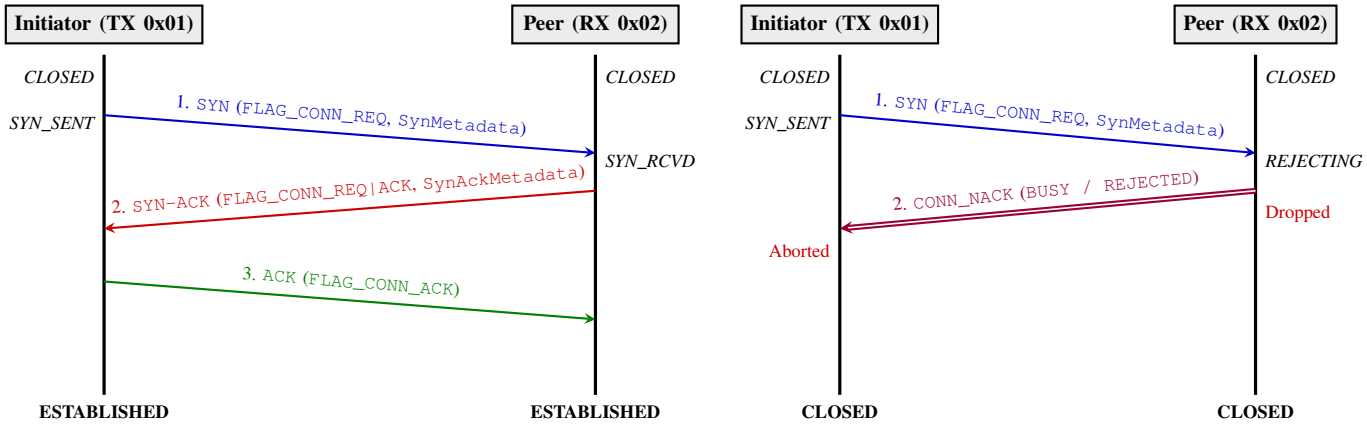
- **Memory and Concurrency Limits**: Governs maximum concurrent reassembly sessions (`MAX_CONCURRENT_SESSIONS` = 10, reducible to 1–2 on memory-constrained Cortex-M0/AVR devices), physical FIFO buffer bounds (`PHY_BUFFER_SIZE` = 256 bytes), and completed sequence deduplication depth (`MAX_COMPLETED_HISTORY` = 16).
- **Timing and Retransmission Policies**: Configures the default retry ceiling ($R_{\max} = 5$), minimum SACK timeout floor ($T_{\min} = 1000$ ms, tunable for low-latency P2P links), safety multiplier ($\alpha = 1.5$), and half-duplex switching guard ($\delta = 25$ ms).
- **Session Lifecycle Thresholds**: Defines handshake initiation timeouts (`DEFAULT_SYN_TIMEOUT_MS` = 3000 ms), connection inactivity teardown periods (`DEFAULT_CONN_INACTIVITY_TIMEOUT_MS` = 15000 ms), and stale session pruning timers (`PRUNE_TIMEOUT_MS` = 15000 ms).



(a) Connection Session Lifecycle FSM (Stateful vs. Stateless)

(b) Data Transfer & SACK-ARQ FSM (Reliable vs. Unreliable)

Fig. 2. Finite State Machine (FSM) specifications for connection session lifecycle and data transfer modes.



(a) Successful 3-Way Handshake (3WHS) with Metadata Negotiation

(b) Connection Rejection (`CONN_NACK`) Failure Mode

Fig. 3. Sequence interaction flows for 3-Way Handshake (3WHS) connection setup with dynamic metadata negotiation and rejection handling.

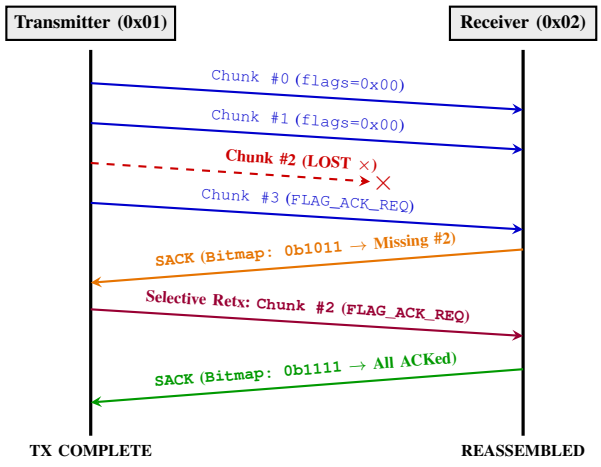


Fig. 4. End-to-end data streaming flow, frame loss detection, and SACK-driven selective retransmission interaction.

Complementing the compile-time configuration, the 3WHS mechanism enables dynamic run-time parameter negotiation (`SynMetadata`), providing a two-tier flexibility model that adapts to variable channel conditions and peer memory capabilities.

B. Hardware Abstraction Layer (HAL)

To keep the protocol core hardware-agnostic, a clean HAL interface (`RadioLibHal`) was implemented. On the physical nodes, the concrete class `EspHal` wraps the ESP32-S3 hardware resources:

- **Serial Peripheral Interface (SPI) Communication**: Manages registers configuration and FIFO buffers transfer for the Semtech SX1262 transceiver.
- **General-Purpose Input/Output (GPIO) Interrupts**: Maps physical pins, including Chip Select (NSS), Reset (RST), Busy status indicator (BUSY), and the hardware interrupt pin (DIO1) used to trigger RX/TX completion interrupts.

- **Timing Primitives**: Implements millisecond counters (`millis()`) and microsecond/millisecond delays (`delay()`).

C. Host-Based Unit Testing

Following a TDD methodology, the protocol core (e.g., packet parsing, reassembler sessions, and CRC checksums) was validated in isolation on a desktop host (Linux/macOS). The tests compile and run natively using the **Unity** testing framework integrated into PlatformIO:

- **Integrity Tests**: Confirmed the Modbus CRC-16 computation on headers and partial/padded payloads.
- **Reassembly State Machine**: Validated reassembly handling of out-of-order and duplicate chunks.
- **Stale Session Pruning**: Confirmed that incomplete reassembly buffers are correctly pruned after exceeding the timeout threshold.

D. Physical Benchmarking Setup

Physical tests were conducted using two Heltec Wi-Fi LoRa 32 V3 development boards. The transmitter node was connected to a console runner to sweep payload sizes, while the receiver node operated as a standalone listener. The transceivers were configured with the following parameters:

- **Carrier Frequency**: 869.525 MHz
- **Spreading Factor (SF)**: 7
- **Bandwidth (BW)**: 500.0 kHz
- **Coding Rate (CR)**: 4/5
- **Output Power**: 10 dBm
- **Preamble Length**: 8 symbols

1) Regulatory Compliance and Reproducibility: It is important to clarify the relationship between the LAMP transport protocol and RF regulatory constraints. LAMP operates strictly at the *transport layer* and is entirely agnostic to duty-cycle regulations, carrier frequency, and RF power limits; compliance with these constraints is the responsibility of the MAC/PHY layer or the integrating application.

For the experimental validation reported in this work, the 869.400–869.650 MHz sub-band was selected [6]. Under ETSI EN 300 220-1, this sub-band permits a maximum duty cycle of 10% and an output power of up to 500 mW (27 dBm), providing sufficient headroom for the multi-chunk burst transfers required by the LAMP reliable delivery modes without incurring the restrictive 1% hourly airtime cap of the primary 868 MHz sub-band.

Additionally, in the reference implementation validated on Heltec V3 hardware, the optional MAC-layer helpers provided by the LAMP library, namely SX1262-native Channel Activity Detection (CAD) with randomized exponential backoff (CSMA-CA) and duty-cycle pacing, were enabled to prevent collisions and further reduce channel occupancy. These features are opt-in and disabled by default; they are not part of the LAMP transport specification but are offered as convenience primitives for integrators targeting shared ISM bands. All experimental results are therefore fully reproducible in any deployment targeting the 869.4–869.65 MHz sub-band with the same radio configuration.

VI. Performance Evaluation and Experimental Methodology

To evaluate the reliability, throughput, and communication limits of LAMP, we adopt a two-stage evaluation methodology: a large-scale stochastic network simulation campaign executed in ns-3, complemented by functional hardware testbed validation on microcontroller nodes and a planned outdoor field benchmarking campaign.

A. Network Simulation Benchmarking (ns-3)

To characterize protocol performance under controlled fading channels across extensive distance ranges, we developed a full C++ simulator integrated into the ns-3 framework (`ns3-lorawan`). The simulator reproduces the exact LAMPv2 frame structure (9-byte Logical Header + 2-byte CRC-16), all four operational modes, the 3WHS handshake, and selective bit-vector SACK retransmissions over a Nakagami-m fast-fading propagation loss model calibrated for the Semtech SX1262 transceiver (+24.2 dBm EIRP, 869.525 MHz).

1) Simulation Campaign Hierarchy: To achieve statistical convergence, an extensive Monte Carlo campaign comprising 117,050 total independent simulation runs was conducted. The experimental space spans 3 RF propagation environments (Line-of-Sight (LoS), Urban, Rural), 3 Spreading Factors (SF7, SF9, SF10), 4 operational modes (Mode 1 to Mode 4), 5 random seeds per parameter set, and 10 payload size steps (scaling from 1 to 100 chunks) across an adaptive distance grid extending dynamically up to 23.5 km with early-stopping thresholds once reception falls to 0% across consecutive steps. Each curve point plotted in Figs. 5 and 6 represents the sample mean over 50 Monte Carlo trials (combining 5 random seeds across 10 payload size iterations per distance step), with shaded ribbons denoting 95% confidence intervals.

2) Analytical Reliability Bound under Stated Assumptions: To model payload completion probabilities analytically, we define a theoretical bound under stated hypotheses:

1) *Independent Loss Model*: Channel frame loss is modeled as an independent and identically distributed (i.i.d.) Bernoulli error process with packet error rate $p \in [0, 1)$. While the i.i.d. Bernoulli model provides a tractable analytical baseline, physical LPWAN channels typically exhibit temporally correlated fading; the stochastic ns-3 simulations evaluated in this work capture these fading dynamics directly without relying on the i.i.d. simplification.
2) *Bounded Retransmissions*: Selective SACK retransmission retries are bounded by the protocol's maximum retry limit $R_{\max} = 5$ (matching `LoRaMultiPacketConfig::MAX_RETRIES`).

Under these stated assumptions, the analytical upper bound for successfully reassembling an N-fragment payload across $R_{\max}$ retries is given by:

$$P_{\text{complete}}(N, p, R_{\max}) = \left(1 - p^{R_{\max}+1}\right)^N \tag{6}$$

It is crucial to clarify that when reliable modes (Mode 3 and Mode 4) sustain a 100% Application Payload Delivery Ratio

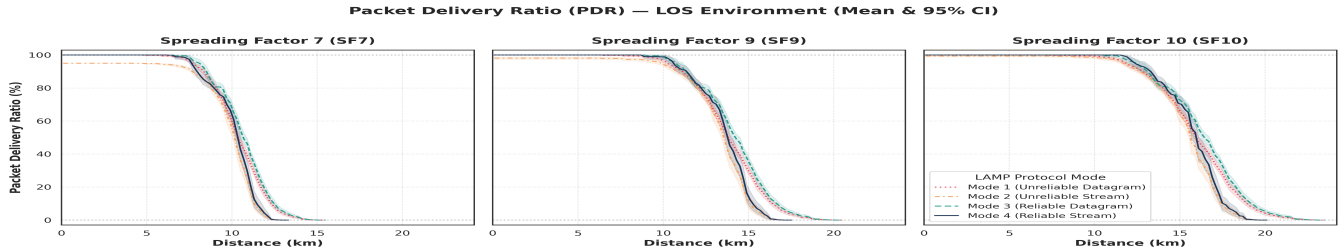


Fig. 5. Packet Delivery Ratio (PDR_{app} %) as a function of distance across Spreading Factors (SF7, SF9, SF10) in Line-of-Sight (LoS) propagation. Solid lines denote mean delivery rates across 50 Monte Carlo trials; shaded ribbons represent 95% confidence intervals.

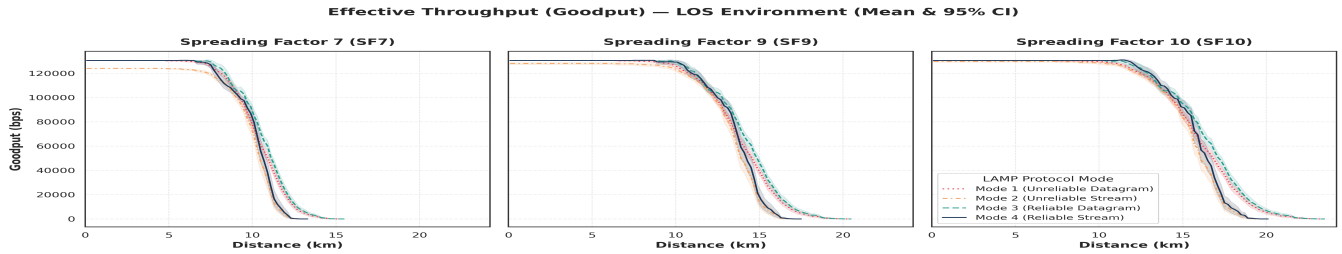


Fig. 6. Effective application-layer throughput (Goodput in bps) across Spreading Factors and LAMP operational modes in LoS. Shaded ribbons denote 95% confidence intervals across 50 Monte Carlo trials.

(PDR_{app}) in Fig. 5, this does not indicate physical channel reliability or zero-loss propagation. Rather, it demonstrates that the selective bit-vector SACK-ARQ mechanism successfully recovers lost fragments within the $R_{\max} = 5$ retry budget prior to session timeout.

In LoS conditions, SF10 sustains complete payload reassembly up to 19.1 km, exhibiting a steep drop-off between 20.0 km and 23.1 km as frame loss probability exceeds retransmission recovery budgets. Conversely, best-effort unreliable modes (Mode 1 and Mode 2) experience progressive packet drop rates as spatial fading degrades signal-to-noise ratios.

Fig. 6 depicts effective application-layer Goodput (bps). While best-effort streaming (Mode 2) achieves peak throughput in pristine channels by omitting acknowledgment latency, Mode 4 (*Reliable Stream*) preserves sustained throughput near physical channel capacity across the operational range, degrading only near receiver sensitivity thresholds.

B. Planned Real-World Field Benchmarking Campaign

While hardware testbed experiments on two Heltec Wi-Fi LoRa 32 V3 nodes have validated the functional correctness, C++ memory safety, 3WHS state machine transitions, and out-of-order reassembly of LAMP, a systematic outdoor field campaign is planned as future experimental work to record real-world empirical performance under environmental interference, multipath reflections, and temperature-induced frequency drift.

1) Planned Experimental Matrix and Evaluation Metrics: The planned outdoor physical validation strategy follows established LPWAN characterization methodologies [8]. It will sweep transmission distances (1 to 5000 meters across LoS and Non-Line-of-Sight (NLoS) conditions) and payload sequence sizes (1 to 100 chunks, $\approx$ 24.6 KB) under standard radio configurations (SF7, BW 500 kHz, CR 4/5, 10 dBm TX power).

During the planned outdoor campaign, four core performance metrics will be recorded across 5 independent trials per operational mode:

- **Application Payload Delivery Ratio** (PDR_{app}): The percentage of application-level payloads successfully reconstructed at the receiver.
- **End-to-End Latency**: Total elapsed duration from initial fragment serialization at the transmitter to final reassembly callback at the receiver.
- **Effective Throughput (Goodput)**: The delivery rate of application payload data, calculated as $\text{PayloadSize (bits)}/\text{Latency (seconds)}$.
- **Retransmission Overhead**: Count of extra fragments and SACK frames transmitted in reliable modes compared to unreliable baselines.

VII. Conclusion and Architectural Roadmap

This paper presents the design, implementation, network simulation, and hardware validation of LAMP, a lightweight and reliable transport-layer architecture optimized for transmitting segmented large payloads over point-to-point and local multi-node LoRa networks. By replacing traditional Stop-and-Wait per-packet acknowledgments with selective retransmissions requested via cumulative binary bitmaps, and eliminating redundant SOM/EOM markers in favor of implicit index derivation, the protocol achieves high reliability under lossy conditions with minimal airtime overhead and duty-cycle footprint.

A. Architectural Roadmap

To guide the long-term progression of LAMP from point-to-point data links to decentralized multiagent mesh networks, we define a strategic three-level architectural roadmap:

- **Level 1 – Addressed Transport & Polite Spectrum Access (Current Implemented Stack)**: The core architecture established in this work. It features 8-bit source and destination node addressing (`srcAddr`, `dstAddr`) with broadcast support (`0xFF`), dynamic 9-byte header packing, four operational modes (unreliable best-effort datagrams to stateful SACK-ARQ streams), a 3WHS for parameter negotiation, and an opt-in HAL helper implementing Channel Activity Detection (CAD)/CSMA-CA with exponential backoff for *Polite Spectrum Access* as specified by ETSI EN 300 220-1 [6].
- **Level 2 – Adaptive Spectrum & Link Optimization (Near-Term Enhancement)**: Extending the link layer to support dynamic channel adaptation in multi-node clusters sharing the same RF channel:
  - *Adaptive Data Rate (Adaptive Data Rate (ADR))*: Dynamically tuning Spreading Factor (SF7–SF12) and Coding Rate (CR) based on real-time RSSI/SNR link quality metrics embedded in SACK feedback frames.
  - *Scheduled Time-Slotted Access*: Incorporating Time Division Multiplexing (TDM) and microsecond time synchronization for collision-free multiagent payload transfers in high-density deployments [13].
  - *Optional Security Profile (Future Work Extension)*: For raw-LoRa multi-node and multi-agent links requiring data privacy, LAMP can be extended with an opt-in Security Profile providing AES-128-GCM or HMAC-SHA256 authentication, payload encryption, and 32-bit sequence replay protection at the transport header level, evaluating byte, latency, and energy overhead in multi-node configurations.
- **Level 3 – Multi-Hop Mesh Extension (Long-Term Vision)**: Scaling the architecture to autonomous multi-hop networks, such as UAV swarms, distributed sensor meshes, and relay networks [11], [12]:
  - *Mesh Routing Protocol Integration*: Combining dynamic routing (e.g., AODV-Light or RPL adaptation) with hop-by-hop vs. end-to-end SACK reliability options to limit retransmission overhead over multi-hop links [12].
  - *IPv6 Network Layer Bridging*: Leveraging the 8-bit Node ID mapping to establish full IPv6 link-local capability (`fe80::/64`), connecting edge nodes directly to global IP infrastructure.

DECLARATIONS

Data Availability: All firmware implementations, C++17 library source code, host unit testing suites, and benchmarking scripts are available in the public project repository.

Ethical Statement & AI Tool Usage: This study did not involve human or animal subjects. AI assistant tools were utilized strictly for code formatting, micro-typography auditing, and manuscript style calibration in compliance with IEEE publication ethics guidelines.

Conflict of Interest: The authors declare that they have no competing financial or personal interests that could influence the work reported in this paper.

REFERENCES

[1] M. Radeta, J. Pestana, P. Abreu, R. Freitas, F. Silva, D. Vieira, R. Prieto, M. Fernandez, F. Alves, T. Dellinger, S. Neves, and E. Delory, "Triton—open telemetry and location estimation for marine monitoring based on iot and lora," *IEEE Journal of Oceanic Engineering*, vol. 50, no. 2, pp. 1244–1258, 2025.

[2] T. Elshabrawy and J. Al-Ali, "Lora technology demystified: From link behavior to cell capacity," *IEEE Transactions on Wireless Communications*, vol. 17, no. 10, pp. 6611–6621, 2018.

[3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the limits of lorawan implementations," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[4] S. De, H. M. Jalajamony, S. Adhinarayanan, S. Joshi, H. Upadhyay, and R. Fernandez, "Multimedia transmission over lora networks for iot applications: A survey of strategies, deployments, and open challenges," *Sensors*, vol. 25, no. 23, p. 7128, 2025.

[5] P. J. Marcelis, V. S. Rao, and R. V. Prasad, "Dare: Data recovery through application layer coding for lorawan," in *Proceedings of the 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2017.

[6] *Short Range Devices (SRD) operating in the frequency range 25 MHz to 1 000 MHz; Part 1: Technical characteristics and methods of measurement*, European Telecommunications Standards Institute (ETSI) Std. ETSI EN 300 220-1 V3.1.1, 2017.

[7] A. Minaburo, L. Toutain, C. Gomez, D. Barthel, and J. C. Zúñiga, "SCHC: Static context header compression and fragmentation for LP-WAN," Internet Engineering Task Force (IETF), RFC 8724, April 2020.

[8] M. Cattani, C. A. Boano, and K. Römer, "An experimental evaluation of the reliability of lora long-range low-power wireless communication," *Journal of Sensor and Actuator Networks*, vol. 6, no. 2, p. 7, 2017.

[9] M. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do lora low-power wide-area networks scale?" in *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2016, pp. 59–66.

[10] T. Chen, D. Eager, and D. Makaroff, "Efficient image transmission using lora technology in agricultural monitoring iot systems," in *2019 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2019, pp. 937–944.

[11] J. Kim, J. Jenkins, J. Seol, and M. Kwak, "Extending data transmission in the multi-hop lora network," *Issues in Information Systems*, vol. 23, no. 3, pp. 292–304, 2022.

[12] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, "Multi-hop and mesh for lora networks: Recent advancements, issues, and recommended applications," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–43, 2024.

[13] C.-C. Wei, J.-K. Huang, C.-C. Chang, and K.-C. Chang, "The development of lora image transmission based on time division multiplexing," in *2020 International Symposium on Computer, Consumer and Control (IS3C)*, 2020, pp. 323–326.

[14] A. Calabrò, E. Marchetti, and M. T. Paratore, "Evaluating use of arq strategies in communication protocols for search and rescue," in *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SCITEPRESS, 2025, pp. 59–70.

[15] S. Corporation, "Lora modem designer's guide," Semtech Corporation, Tech. Rep. Application Note AN1200.13, 2015.